

Agentic Architect Challenge

Developer Intern Assessment | Sean Lee Jia ROUNG

1. System Overview

The solution uses LLM reasoning only where semantic interpretation is useful, while deterministic code controls objective rules, arithmetic, limits, retries, and validation. Across the three parts, the design follows the same reliability principle: the model may reason and choose tools, but important outputs are grounded in retrieved evidence or verified by deterministic components before they are accepted.

2. Part 1 : Customer Support Email System

Incoming emails are first classified into categories such as Billing, Technical, and Feedback. Before any normal response is drafted, an escalation guardrail checks for possible data loss, service outage, security breach, or more than three support contacts within seven days. Semantic conditions are identified by the LLM, while support-history counts are determined from stored customer records. Non-escalated requests use retrieval-augmented generation (RAG) over approved PDFs and FAQs. A response agent drafts from the retrieved evidence, and a grounding/policy validator checks material claims before release. Unsupported refund-policy facts are never guessed; insufficient or contradictory evidence is routed to human review.

3. Part 2 : Web Summarizer

The summarizer first fetches static HTML with Jsoup because it is lightweight and fast. After cleaning navigation, hidden content, and other noise, the system evaluates whether meaningful content was extracted. Sparse or JavaScript-rendered pages fall back to Playwright/Chromium. Long content is split into bounded chunks, summarized individually, and combined through a map-reduce process before final synthesis. A deterministic guardrail enforces a maximum 150-word summary; an over-limit result receives bounded compression attempts and then fails safely rather than being blindly truncated. Network failures, malformed URLs, browser timeouts, and model failures are handled with bounded retries and clear errors.

4. Part 3 : Document Agent

The document agent is a conversational CLI backed by a single fictional handbook. The LLM chooses structured tools rather than Java routing user messages by keywords. It can call `document_search` for handbook evidence and calculator for deterministic arithmetic, including sequential use when a calculation depends on a retrieved policy value. Conversation memory retains bounded completed turns and grounded evidence, allowing follow-up questions such as calculating several years of a previously retrieved allowance. Final answers validate evidence IDs and calculation provenance. If the model attempts a derived calculation without a current deterministic calculator result, the agent provides one bounded recovery opportunity before failing safely.

5. Trade-offs and Failure Points

The main trade-off is reliability versus latency and API cost: retrieval, validation, browser fallback, and recovery add work, but reduce unsupported outputs. Part 3 uses lexical retrieval because a single sample document does not justify embedding infrastructure; this is simple and explainable but can miss semantic paraphrases. LLM tool choice remains probabilistic, so bounded validation and recovery are required. External dependencies can also fail: knowledge retrieval, web rendering, or model APIs may be unavailable. The system therefore uses timeouts, bounded retries, structured validation, safe error paths, and human escalation rather than silently inventing an answer. Logging records operational events and reason codes without exposing API keys or unnecessary conversation content.